Real Estate Support Triage Agent Report

Course Name: Agentic AI

Medicaps University

Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr no	Student Name	Enrolment Number
1	Sandesh Vani	EN22CS301862
2	Shivam Choudhary	EN22CS301903
3	Shreya Gupta	EN22CS301927
4	Shrishti Chouksey	EN22CS301935
5	Shyam Narayan Pandey	EN22CS301949
6	Siddhi Suraniya	EN22CS301956

Group Name: 12 D8

Project Number: AAI-47

Industry Mentor Name: Mr. Suraj Nayak

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2026

Problem Statement and Objectives

1. Problem Statement

The Real Estate Support Triage Agent is an AI-based system designed to automate the initial handling of customer communications in the real estate industry. Real estate companies receive a large number of messages daily through emails, CRM portals, websites, and messaging platforms. These messages may relate to booking confirmations, payment issues, maintenance complaints, legal document requests, or site visit rescheduling. Traditionally, support staff manually read each message, identify the issue, determine its urgency, extract important details like booking ID or payment date, and draft a response. This manual process is time-consuming, increases operational costs, and may lead to delays or errors, especially when the volume of inquiries is high.

The main problem is the lack of an intelligent system that can automatically understand and process unstructured customer messages in real time. Customers often use informal language, mixed languages, or incomplete information, making it difficult for simple rule-based systems to interpret their requests. Additionally, not all issues have the same priority. For example, a maintenance complaint such as water leakage requires urgent attention, while a general inquiry about project details can be handled later. Therefore, accurate intent classification and urgency detection are essential.

To solve this problem, the proposed system uses Large Language Models (LLMs) to analyze incoming messages. The system classifies the intent, determines urgency, extracts key entities like booking ID and dates, and generates a professional draft response. It can also route the request to the appropriate department. This automation reduces response time, improves efficiency, ensures consistent communication, and enhances overall customer satisfaction in real estate support operations.

2. Problem Objectives

- **To automate customer communication processing**
  The main objective of this project is to automate the initial handling of customer messages in the real estate domain using Artificial Intelligence.
- **To reduce manual workload**
  The system aims to minimize the dependency on human support agents by automatically analyzing and responding to common queries.

- **To classify customer intent accurately**
  The project focuses on identifying the intent of incoming messages, such as payment issues, booking inquiries, maintenance complaints, legal requests, or rescheduling requests.
- **To determine urgency levels**
  The system should categorize messages into high, medium, or low urgency so that critical issues are prioritized and resolved faster.
- **To implement Named Entity Recognition (NER)**
  The agent will extract important details like booking ID, flat number, payment amount, and dates from unstructured text messages.
- **To generate professional draft responses**
  The system will use a Large Language Model (LLM) API to create accurate and polite draft replies for customers.
- **To enable intelligent ticket routing**
  Based on classification and urgency, the system will forward queries to the appropriate department automatically.
- **To improve response time and efficiency**
  The project aims to significantly reduce response delays and improve overall operational efficiency.
- **To ensure scalability and reliability**
  The system should be capable of handling a high volume of messages and work effectively in real-time environments.
- **To enhance customer satisfaction**
  By providing faster, consistent, and accurate responses, the project aims to improve the overall customer experience in real estate support services.

3. Scope of the Project

- **Automation of Initial Support Handling**
  The project focuses on automating the first level of customer support in the real estate sector by analyzing incoming messages and generating appropriate responses.
- **Message Classification and Prioritization**
  The system will classify customer queries based on intent (payment, booking, maintenance, legal, etc.) and assign urgency levels (high, medium, low) to ensure proper prioritization.
- **Entity Extraction from Unstructured Text**
  The project includes implementing Named Entity Recognition (NER) to extract important details such as booking IDs, flat numbers, dates, and payment amount from customer messages.

- **Automated Draft Response Generation**
  The system will generate professional and context-aware draft replies using a Large Language Model (LLM) API to assist support teams.
- **Integration with CRM or Support Systems**
  The solution can be integrated with existing CRM platforms or helpdesk systems to automatically create and route tickets.
- **Real-Time Processing**
  The agent will process incoming messages in real time, ensuring faster response and efficient handling of customer queries.
- **Scalability for High Message Volume**
  The system is designed to handle a large number of messages without performance degradation.
- **Support for English and Mixed-Language Inputs**
  The project scope includes handling informal or mixed-language customer messages commonly used in real-world communication.
- **Security and Data Privacy Measures**
  The system will ensure safe handling of customer information and maintain data confidentiality.
- **Limitations**
  The project scope is limited to initial triage and draft generation. Final approval of responses and complex decision-making may still require human intervention.

Proposed Solution

1. Key Features

- Real-time message processing
- Intent classification
- Urgency detection
- Named Entity Recognition (NER)
- Automated draft response generation
- Intelligent ticket routing
- Intelligent ticket routing
- CRM integration capability
- Scalable and production-ready architecture
- Data security and privacy handling
- Confidence scoring and human review option

2. Overall Architecture and Workflow

Architecture

The Real Estate Support Triage Agent follows a modular and scalable architecture designed for real-time message processing and intelligent automation. The system consists of multiple interconnected components that work together to analyze, classify, extract, and respond to customer communications efficiently.

The architecture begins with the **Input Layer**, where customer messages are received from various channels such as email, CRM systems, website forms, or messaging platforms. These messages are then passed to the **Preprocessing Layer**, where basic text cleaning and formatting are performed.

Next, the message is sent to the **Intent Classification and Urgency Detection Module**, powered by a Large Language Model (LLM). This module determines the purpose of the message (such as payment issue, booking inquiry, maintenance request) and assigns an urgency level (high, medium, or low).

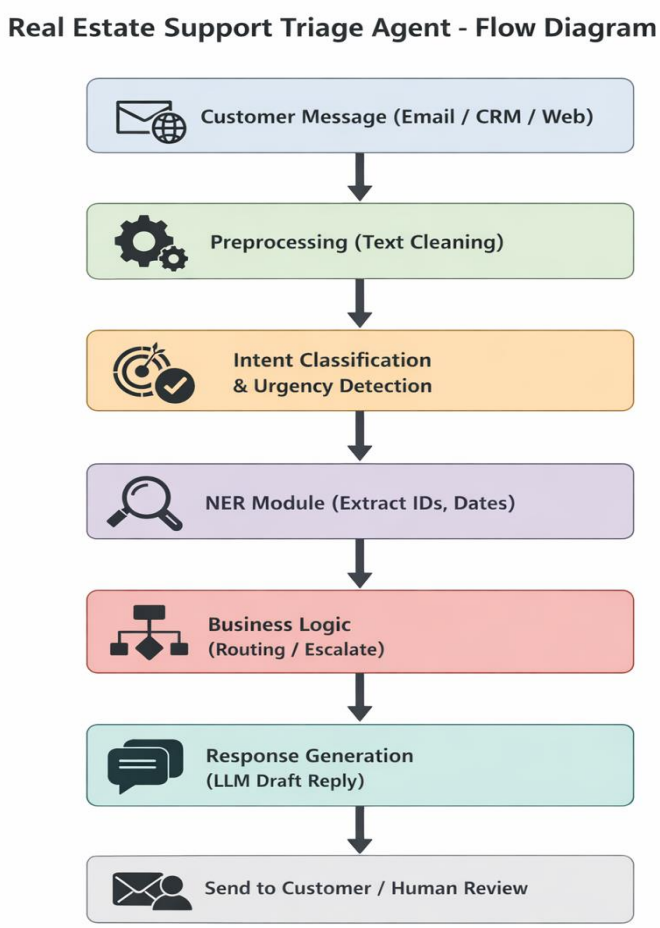
After classification, the message is processed by the **Named Entity Recognition (NER) Module**, which extracts key information such as booking ID, flat number, payment amount, and dates. The extracted structured data is then forwarded to the **Business Logic Layer**, which decides routing rules and escalation actions.

Finally, the **Response Generation Module** uses the LLM API to create a professional draft reply. The output is then stored in the database and optionally sent to the customer or forwarded to a support agent for review.

Workflow

- Customer sends a message.
- Message is received and pre-processed
- LLM classifies intent and urgency.
- NER extracts important details.
- Business logic determines routing and escalation.
- LLM generates draft response.
- Response is sent or reviewed by human agent.

This structured workflow ensures faster processing, intelligent routing, and improved customer support efficiency.



3. Tools and Technologies Used

1. Python

- Primary programming language used for system development.
- Handles backend logic, API integration, and data processing.
- Supports NLP libraries and web frameworks.

2. Fast API / Flask (Optional Backend Framework)

- Used to build REST APIs for real-time message processing.
- Enables scalable and production-ready deployment.

3. Lang Chain

- Framework for building LLM-based pipelines.
- Used to chain tasks like classification, NER, and response generation.

- Helps manage prompts and workflow orchestration.

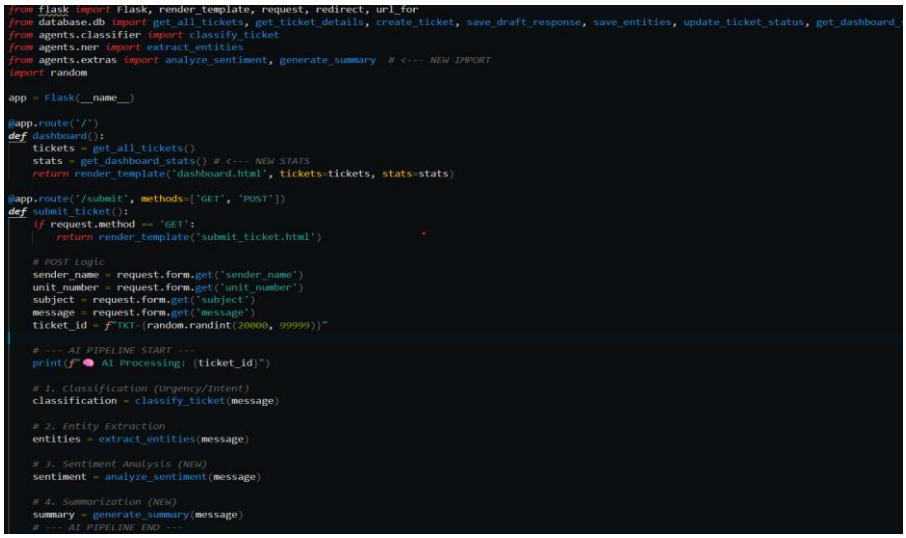
4. **Crew AI**
   - Multi-agent framework for collaborative AI agents.
   - Separate agents can handle intent detection, entity extraction, response drafting, and escalation.
   - Improves modularity and system structure.
5. **LLM API (Large Language Model API)**
   - Core intelligence of the system.
   - Performs intent classification and urgency detection.
   - Extracts structured data from unstructured text.
   - Generates professional draft responses.
6. **Named Entity Recognition (NER)**
   - Extracts key information like booking ID, dates, and payment amounts.
   - Can be implemented using LLM or NLP libraries.
7. **Database (Optional – MySQL / PostgreSQL)**
   - Stores messages, classification results, and responses.
   - Maintains ticket records and logs.
8. **Version Control (Git/GitHub)**
   - Used for source code management and collaboration.

These tools together ensure scalability, automation, and intelligent processing of customer communications.

Results and Outputs

1. Code Screenshots

- **Main Code**



```
from flask import Flask, render_template, request, redirect, url_for
from database.db import get_all_tickets, get_ticket_details, create_ticket, save_draft_response, save_entities, update_ticket_status, get_dashboard_
from agents.classifier import classify_ticket
from agents.ner import extract_entities
from agents.extras import analyze_sentiment, generate_summary  # <--- NEW IMPORT
import random

app = Flask(__name__)

@app.route('/')
def dashboard():
    tickets = get_all_tickets()
    stats = get_dashboard_stats() # <--- NEW STATS
    return render_template('dashboard.html', tickets=tickets, stats=stats)

@app.route('/submit', methods=['GET', 'POST'])
def submit_ticket():
    if request.method == 'GET':
        return render_template('submit_ticket.html')

    # POST Logic
    sender_name = request.form.get('sender_name')
    unit_number = request.form.get('unit_number')
    subject = request.form.get('subject')
    message = request.form.get('message')
    ticket_id = f"TKT-{random.randint(20000, 99999)}"

    # --- AI PIPELINE START ---
    print(f"🧠 AI Processing: {ticket_id}")

    # 1. Classification (Urgency/Intent)
    classification = classify_ticket(message)

    # 2. Entity Extraction
    entities = extract_entities(message)

    # 3. Sentiment Analysis (NEW)
    sentiment = analyze_sentiment(message)

    # 4. Summarization (NEW)
    summary = generate_summary(message)
    # --- AI PIPELINE END ---
```

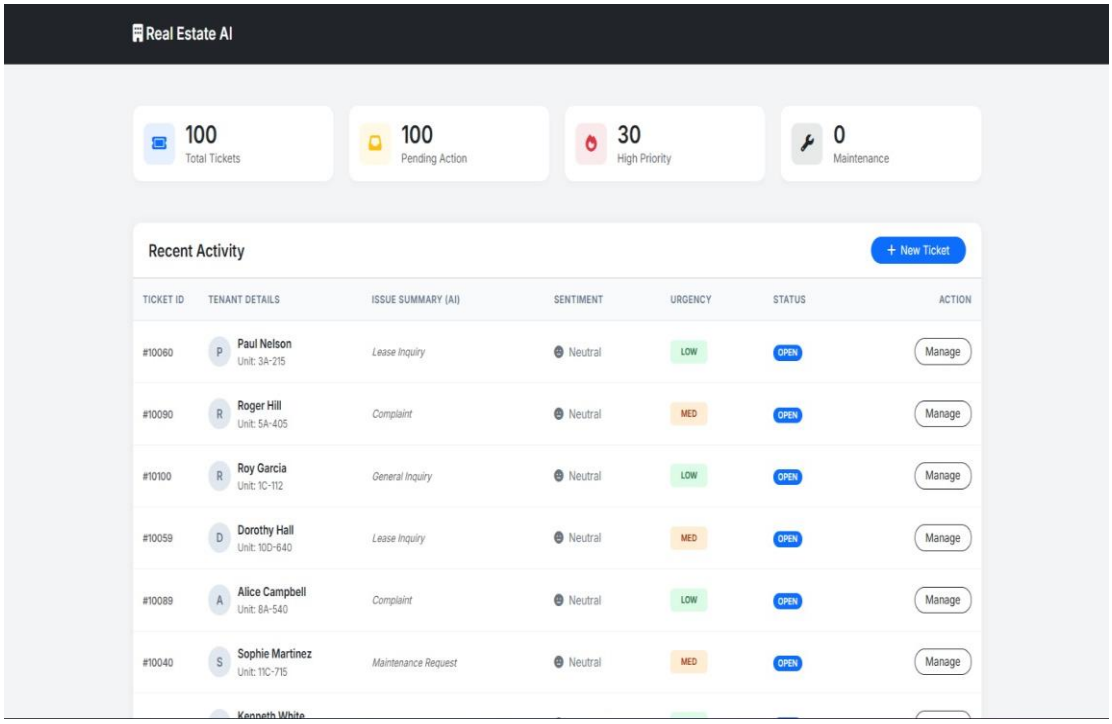
- **Agents**

```python
import os
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langchain_core.prompts import PromptTemplate

load_dotenv()

llm = ChatGroq(
    model="llama-3.3-70b-versatile",
    temperature=0,
    groq_api_key=os.getenv("GROQ_API_KEY")
)

# --- AGENT 3: SENTIMENT ANALYZER ---
sentiment_template = """
Analyze the sentiment of this tenant message.
Return ONLY one word: [POSITIVE, NEUTRAL, NEGATIVE, ANGRY].

Message: "{message}"
"""
sentiment_prompt = PromptTemplate(input_variables=["message"], template=sentiment_template)

def analyze_sentiment(message):
    chain = sentiment_prompt | llm
    return chain.invoke({"message": message}).content.strip().upper()

# --- AGENT 4: SUMMARIZER ---
summary_template = """
Write a very short 5-8 word summary of this issue for a dashboard view.
Example: "Leaking sink in kitchen"
Message: "{message}"
"""
summary_prompt = PromptTemplate(input_variables=["message"], template=summary_template)

def generate_summary(message):
    chain = summary_prompt | llm
    return chain.invoke({"message": message}).content.strip()
```

- **Database**

```python
def get_db_connection():
    return mysql.connector.connect(
        host=os.getenv('DB_HOST'),
        user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'),
        database=os.getenv('DB_NAME')
    )

def get_all_tickets():
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM support_tickets ORDER BY created_at DESC")
    tickets = cursor.fetchall()
    conn.close()
    return tickets

def get_ticket_details(ticket_id):
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM support_tickets WHERE ticket_id = %s", (ticket_id,))
    ticket = cursor.fetchone()
    if ticket:
        cursor.execute("SELECT draft_text FROM draft_responses WHERE ticket_id = %s ORDER BY id DESC LIMIT 1", (ticket_id,))
        draft = cursor.fetchone()
        ticket['draft_response'] = draft['draft_text'] if draft else "No draft generated."

        cursor.execute("SELECT entity_type, entity_value FROM extracted_entities WHERE ticket_id = %s", (ticket_id,))
        ticket['entities'] = cursor.fetchall()
    conn.close()
    return ticket

def create_ticket(data):
    conn = get_db_connection()
    cursor = conn.cursor()
    query = """
    INSERT INTO support_tickets
    (ticket_id, sender_name, sender_email, unit_number, subject, message, category, urgency, sentiment, summary, status)
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, 'open')
    """
    cursor.execute(query, (
        data['ticket_id'], data['sender_name'], data['sender_email'],
```

2. Output



3. Key Outcomes

- **Faster Response Time**
  The system processes and analyzes customer messages in real time, significantly reducing the time required to generate responses compared to manual handling.
- **Reduced Manual Effort**
  Automation of message classification, entity extraction, and draft generation minimizes the workload on support agents and allows them to focus on complex cases.
- **Accurate Intent Identification**
  The AI model effectively classifies customer queries into categories such as payment issues, booking inquiries, maintenance complaints, and legal requests.
- **Effective Priority Management**
  Urgency detection ensures that high-priority issues are identified and escalated immediately, improving service reliability.
- **Structured Information Extraction**
  The system extracts key entities like booking IDs, flat numbers, dates, and payment amount from unstructured text, improving data organization.

- **Professional Draft Responses**
  The LLM generates context-aware, polite, and standardized draft replies, ensuring consistency in communication.
- **Intelligent Ticket Routing**
  Queries are automatically forwarded to the appropriate department based on classification and urgency level.
- **Improved Operational Efficiency**
  Automation reduces delays, enhances workflow management, and increases overall productivity.
- **Scalable Architecture**
  The system is capable of handling a large volume of customer messages without performance degradation.
- **Enhanced Customer Satisfaction**
  Faster responses, accurate handling, and consistent communication lead to a better overall customer experience.
- **Reduced Human Errors**
  Automated data extraction and classification minimize mistakes caused by manual processing.

Conclusion

The Real Estate Support Triage Agent project successfully demonstrates the practical implementation of Artificial Intelligence in automating customer support operations within the real estate domain. Through this project, an intelligent system was developed using Python, Lang Chain/Crew AI, and a Large Language Model (LLM) API to analyze incoming customer messages, classify their intent and urgency, extract key entities, and generate professional draft responses. The system effectively converts unstructured text into structured, actionable information, enabling faster processing and intelligent ticket routing.

The project highlights how AI-driven automation can significantly reduce manual workload, improve response time, and enhance overall customer satisfaction. By integrating multiple components such as intent classification, Named Entity Recognition (NER), business logic handling, and response generation into a single workflow, the solution ensures efficiency, scalability, and consistency in communication.

The key learnings from this project include understanding how Large Language Models work in real-world applications, designing modular system architectures, implementing multi-agent workflows using Lang Chain or Crew AI, and building API-based automation systems. Additionally, it provided practical exposure to handling unstructured data, designing scalable backend systems, and applying AI to solve business problems.

Overall, this project strengthened technical knowledge in AI integration and system design while demonstrating the value of intelligent automation in improving operational efficiency in the real estate industry.

Future Scope and Enhancements

1. **Multilingual Support**
   The system can be enhanced to support multiple languages such as Hindi and regional languages to handle a wider customer base.
2. **Voice-Based Integration**
   Integration with voice assistants or call centre systems to process voice queries using speech-to-text technology.
3. **Advanced Analytics Dashboard**
   Development of a real-time dashboard to monitor ticket volume, urgency distribution, response time, and system performance.
4. **Sentiment Analysis Integration**
   Adding sentiment detection to identify frustrated or dissatisfied customers and prioritize their queries.
5. **Auto-Reply with Human Override**
   Implementing a fully automated reply system with an option for human review before sending responses.
6. **CRM Deep Integration**
   Seamless integration with enterprise CRM systems for automatic ticket creation, tracking, and status updates.
7. **Learning from Feedback**
   Incorporating a feedback loop where the system improves classification accuracy based on human corrections.
8. **AI-Based Escalation Prediction**
   Predicting potential escalation cases based on historical patterns and customer behaviour.
9. **Enhanced Security Measures**
   Implementing advanced data encryption and compliance with data protection regulations.
10. **Performance Optimization**
    Reducing API response latency and optimizing token usage to lower operational costs.
11. **Knowledge Base Integration**
    Connecting the system with a company knowledge base to provide more accurate and detailed responses.